面向边缘智能的

多范式神经网络 FPGA 加速平台

设 计 报 告

匿名评审材料：不含学校、团队与个人信息

目 录

1 选题背景与创新点
1.1 应用背景与实际问题
1.2 已有工作与差距
1.3 本作品的创新点
1.4 主要指标速览
2 设计原理与功能框图
2.1 总体架构
2.2 CNN 库：ONNX → 指令流 / RTL / IP
2.3 SNN 库：配置 + checkpoint → 全整数流水线
2.4 LNN 库：CfC 单步图 → 定点 RTL
2.5 统一数值合同
3 软硬件划分依据与接口设计
3.1 划分依据
3.2 硬件接口与地址映射
3.3 软件接口
3.4 时钟、复位与 CDC
4 优化过程与性能资源对比
4.1 方法与迭代
4.2 CNN 库优化
4.3 SNN 库优化
4.4 LNN 库优化
4.5 三库最终指标
5 大模型协作记录
5.1 协作模式与工作流
5.2 提示词与指令设计
5.3 自我纠错与人工把关
5.4 协作成效与边界
6 技能包的提炼过程
6.1 从失败到规则
6.2 技能包结构与内容
6.3 有效性验证
6.4 边界与失效条件
7 复现说明
7.1 环境要求
7.2 从零开始步骤
7.3 常见问题
7.4 证据核对清单
附录 A 工程仓库目录结构（3.3.5.4 映射）
附录 B 缩略语

1 选题背景与创新点

1.1 应用背景与实际问题

边缘智能设备越来越需要在同一台设备上同时处理图像、文本和时序信号。图像侧以卷积网络（CNN）为主，文本侧出现了脉冲神经网络（SNN）与 Transformer 结合的方案，时序控制侧则有液态神经网络（LNN）这类连续时间模型。三类模型的计算形态差异很大：CNN 是密集卷积与定点乘加，SNN 是稀疏 0/1 脉冲与位运算，LNN 是小规模状态递推与非线性查表。

工程上，这带来三个具体问题。第一，每个模型通常要单独做一套加速器，模型一换就要重做 RTL、重写驱动、重跑验证；第二，从模型到板卡的链路是割裂的——训练框架、量化工具、RTL 生成、Vivado 工程、板端脚本各自为政，中间没有统一的数值合同与身份绑定；第三，验证结果难以复核，仿真通过不等于综合通过，综合通过不等于上板正确，历史结果和本次结果经常被混在一起。我们在自己的开发过程中就多次踩到这类问题：把核级 OOC 结果当成平台完成、把仿真模型输出当成板端实测、被一个没有依据的资源门槛卡住数天。

本作品要解决的实际问题是：在同一颗 AMD ZU15EG 上，用一套可复现的转译与证据方法，把 CNN、SNN、LNN 三类模型都送进 PL 并跑通，让换模型只需要重新转译或重新导出，而不是重做硬件；让每一个结论都能追溯到具体文件。

1.2 已有工作与差距

CNN 的 FPGA 加速已经比较成熟：Xilinx Vitis AI、HLS 库、以及各类指令流 NPU 都能把 ONNX 映射到 PL；但这些方案通常绑定单一工具链与单一量化格式，算子覆盖与调度策略不透明，换模型时需要重新适配。

SNN 方面，Spikformer 把 Transformer 的注意力改成脉冲形式，已有工作多集中在算法与仿真层面；面向 FPGA 的整链实现（参数打包、RTL 配置、时序收敛、板端运行）仍不多见，且全整数数值合同往往没有写清楚。

LNN 方面，CfC（Closed-form Continuous-time）网络 2022 年提出，硬件实现很少；已有实现多为小规模仿真，缺少把训练权重定点化、写成 RTL ROM 初值并逐位核对的完整链路。

与上述工作相比，本作品的差异不在单个模型的速度，而在三件事：一是三类模型共用一套“模型—数值合同—转译—验证—证据”的方法；二是转译产物自带身份（源码哈希、参数清单、数值合同版本），可以复算；三是所有结论按 pass / fail / partial / not_run 分级，并给出原始证据路径。

1.3 本作品的创新点

· 提出“三范式一平台”的统一转译与证据方法：CNN、SNN、LNN 共用数值合同、源码哈希绑定与分级证据记录，换模型只重编译或重导出，RTL 库不动。

· CNN 采用编译期指令流 + 图感知引擎裁剪：按模型图决定物理引擎实例，未用引擎在 elaboration 阶段裁掉，平台 LUT 降低 26.9%、DSP 降低 10.6%，ABI 1.0 保持不变。

· SNN 实现全整数 Spikformer 变体与 5 级 MAC 流水：把 LIF、QK popcount、未归一 AV、残差只加 0/1 脉冲写进数值合同，在 200 MHz（5 ns）下 OOC 与平台均 0 失败端点。

· LNN 把 CfC 单步图整单元映射为单 MAC 时分复用结构：CfC 核仅 1 个 DSP48E2，64 步 9088 拍，训练权重经单步 ONNX 与 int16 Q12 定点导出后直接成为 ROM 初值。

· 用大模型智能体完成工程闭环，并把失败沉淀为技能包：任务书、交接规范、证据纪律与 4 个可执行技能，使他人可以按文档复现本作品。

1.4 主要指标速览

表 1 汇总三个库的代表模型、数值格式、资源与时序。资源与时序均取自仓库实测记录；CNN 为历史 r4 与 16/8 两套实现，SNN 为 OOC 与平台 r7，LNN 为最终位流实现。

表 1 三个库的主要指标

项目	CNN 库	SNN 库	LNN 库
代表任务	YOLOv8n-ReLU VOC20 检测	Spikformer 中文情感分类	CfC 正弦延迟 4 步
输入	静态 INT8 QDQ ONNX	配置 JSON + 整数 checkpoint	训练权重 + 配置
数值格式	affine-int8（int8 权重/激活，int32 累加）	全整数（LIF 33 位扩展、QK 位与 popcount、AV 不归一）	int16 Q12 / int32 Q24 / 48 位累加
时钟	200 MHz（5 ns）	200 MHz（5 ns）	300 MHz（3.33 ns）

项目	CNN 库	SNN 库	LNN 库
片上资源	LUT 140,028 / DSP 471（r4）；106,531 / 1,203（16/8）	LUT 40,964 / DSP 128（OOC）；72,167 / 131（平台 r6）	LUT 25,221 / DSP 4（全设计）；2,189 / 1（CfC 核）
时序	WNS +0.075 / WHS +0.010（r4）	OOC WNS +0.135 / WHS +0.023；平台 r7 +0.051 / +0.011	WNS +0.226 / WHS +0.011
关键结果	103 条指令；16/8 板端 51.4 ms（PS 20 + PL 31）	冻结 RTL 回放 0 mismatch；板端 3/3（记录级）	64/64 输出与末状态逐位一致；导出重算 7/7 逐字节一致

2 设计原理与功能框图

2.1 总体架构

系统分为 PC 侧转译工具链与 ZU15EG 平台两侧。PC 侧负责模型训练/量化、数值合同、转译生成、仿真与时序门禁和证据归档；平台侧由 PS（ARM Cortex-A53 ×4 + PYNQ Linux）与 PL 组成。PS 通过 AXI-Lite 访问各加速器的寄存器窗口，完成任务配置、启动与状态轮询；通过 AXI-Full/DMA 与 PL DDR4 搬运参数与数据；PL 内并列部署 CNN、SNN、LNN 三个加速库，各自占用独立的 AXI 主口与存储窗口。

三个库共用同一套交付形态：每次转译生成 manifest 与工程文件（参数、RTL 配置、约束、综合/实现入口），经仿真与时序门禁后生成位流，再由 PYNQ 驱动在板上运行并逐位比对；所有结果按证据路径归档。图 1 给出系统总体框图。

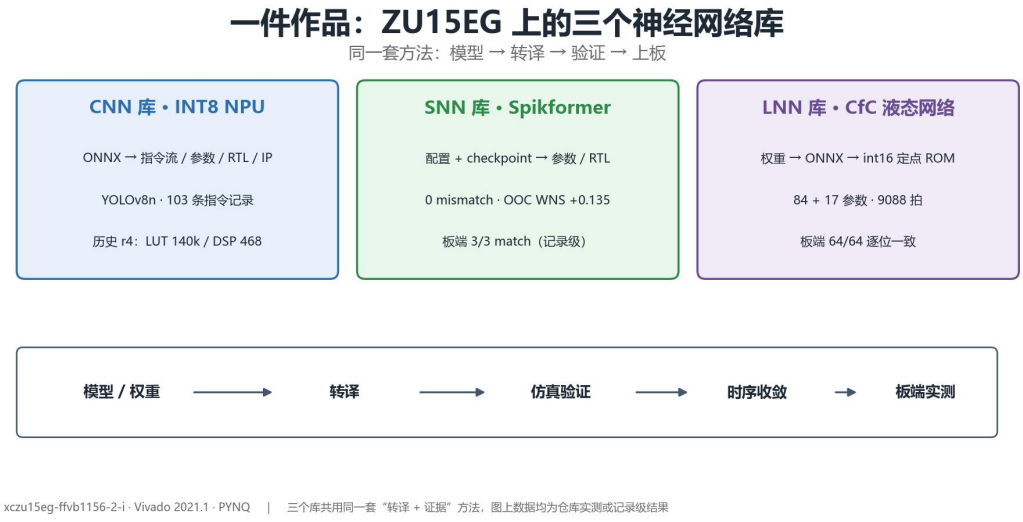


图 1 系统总体框图（三库 + 统一转译与证据流程）

2.2 CNN 库：ONNX 指令流 / RTL / IP

CNN 编译器以静态 INT8 QDQ ONNX 图为输入，先做图解析与量化合同检查，再做图降低、算子规划与指令编码，最后生成整图指令流、打包参数、DDR 初始化镜像和模型专用 RTL 顶层，并由 IP 打包器输出 Vivado IP Catalog 源码包。编译器不启动 Vivado，物理实现由构建脚本调用。

硬件侧是一个指令流 NPU：从 PS DDR 读取指令与参数，按指令驱动 11 类算子核（dense conv、depthwise、maxpool、resize、concat、LUT 激活等），结果通过 AXI 主口写回。ABI 1.0 规定寄存器布局与 128 bit 存储协议，因此模型更换只需重新编译，不需要改 RTL 与驱动。

数值方面采用 affine-int8-v1：int8 对称量化权重与激活、int32 累加、按张量 scale 反量化到浮点参考；量化误差与整型溢出在编译期检查，越界直接报错而不是静默截断。图 2 给出 CNN 的转译与执行流程。



图 2 CNN 库转译与执行流程

2.3 SNN 库：配置 + checkpoint → 全整数流水线

SNN 工具链以配置 JSON 与整数 checkpoint 为输入，按配置推导几何参数（E、heads、head_dim、T、序列长度、深度、MLP 宽度），完成参数打包、RTL 结构配置、目标顶层生成、约束与片上调度生成；compile / build / package 三段式分别对应参数与 RTL 配置、工程与实现入口、板端运行包。

硬件侧是一个 5 级 MAC 流水线的 Spikformer block：QKV 脉冲线性层 → QK 位与 + popcount → 未归一 AV → proj + 残差 → MLP + 残差，DEPTH 个 block 由序列控制器切换权重基址。数值合同是全整数：LIF 采用 (pre + delta) >>> 1 并做 33 位符号扩展，QK 不除 head_dim，AV 对 TOKENS 求和且不归一，残差只加 0/1 脉冲；head 归约在 PL 做每通道求和，PS 侧做唯一的 FP32 线性层与 argmax。

图 3 给出 SNN 从真实语料到板端结论的完整链路。

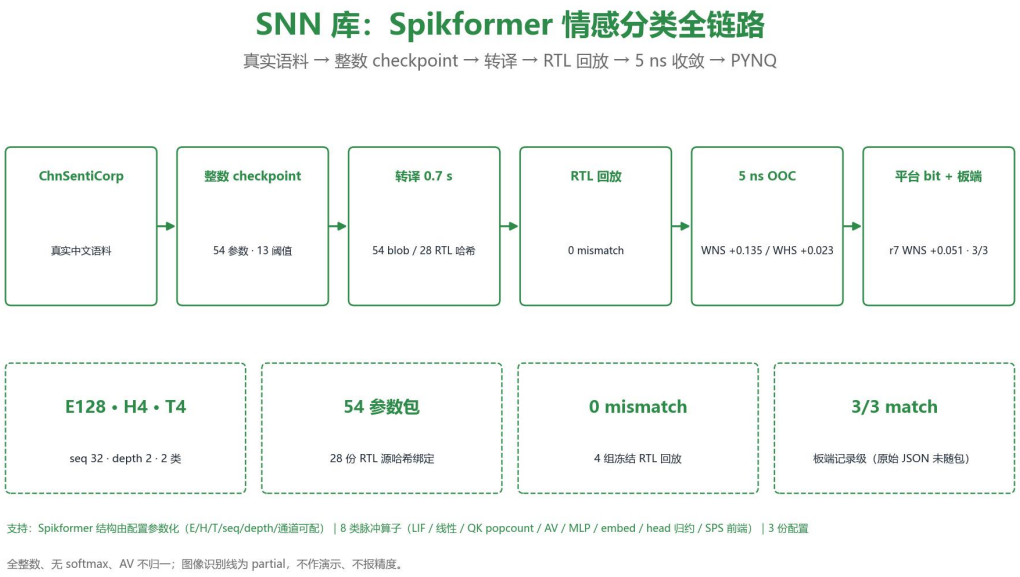


图 3 SNN 库全链路（语料 → 整数 checkpoint → 转译 → 回放 → 板端）

2.4 LNN 库：CfC 单步图 → 定点 RTL

LNN 库面向 1 输入 / 4 隐状态 / 1 输出的 CfC 单元。训练好的浮点权重先导出为单步 ONNX（opset 17，输入 x/h/Δt，输出 y/h_next），ONNX 与 PyTorch 逐点比对通过后，导出器按数值合同把权重转成 int16 Q12、偏置转成 int32 Q24、tanh 与 sigmoid 各 1025 点 LUT，并生成 RTL 的 ROM 初始化头文件。

硬件侧 8 个 RTL 模块：MAC 阵列（单 MAC 时分复用）、权重/偏置 ROM、激活 LUT ROM、状态寄存器、cell 核状态机、序列控制、AXI-Lite 寄存器与平台 wrapper。每步 142 拍、64 步共 9088 拍，300 MHz 下约 30.3 μs；换任务只需重训权重并重新导出 ROM 初值，RTL 与位流结构不变。图 4 给出 LNN 的定点链路。



图 4 LNN 库定点转译链路

2.5 统一数值合同

三个库的数值合同都写进 manifest，并由同一份整数参考在 Python、寄存器模型与 RTL 三处复核。表 2 给出对照。

表 2 三个库的数值合同对照

项目	CNN 库	SNN 库	LNN 库
权重	int8 per-tensor 对称	int8 per-tensor 对称	int16 Q12
偏置	int32	int32	int32 Q24
累加	32 位	32 位（LIF 33 位扩展）	48 位
激活	int8 饱和 + LUT	0/1 脉冲、硬复位	tanh / sigmoid 1025 点 LUT
特殊规则	QDQ 前端、per-tensor scale	无 softmax；AV 不归一；残差只加 0/1	Δt=655（Q12）、round-half-away、饱和
复核方式	ModelSim 回放 0 mismatch	RTL 回放 0 mismatch	RTL 逐位仿真 + 板端 64/64

3 软硬件划分依据与接口设计

3.1 划分依据

软硬件划分遵循三条依据。第一，计算形态：密集、规则、可并行的乘加放进 PL；分支多、访存不规则、需要浮点或字符串处理的部分留在 PS。第二，数据规模：参数与中间结果超过片上存储时走 DDR，PL 通过 AXI 主口或 DMA 访问，PS 只负责搬运控制与结果整理。第三，可验证性：每个库必须有一个“整数参考”能在 PS 侧独立复算，PL 输出必须能与这个参考逐位比对，否则不进入上板流程。

按此划分，CNN 的算子阵列、指令译码与 DDR 读写全部在 PL，编译器与结果解析在 PS；SNN 的脉冲算子、block 流水与 wrapper 在 PL，tokenize、head 的 FP32 线性层与 argmax 在 PS；LNN 的 CfC 数据通路与 ROM 在 PL，导出器、驱动与比对在 PS。表 3 给出对照。

表 3 三个库的软硬件划分

库	PL（硬件）	PS（软件）	划分依据
CNN	指令流 NPU、11 类算子核、DDR 读写、CSR	ONNX 编译器、IP 打包、PYNQ 驱动、结果解析	密集乘加与规则访存；模型图在编译期确定
SNN	5 级 MAC 流水 block、脉冲算子、序列控制器、wrapper、DDR 窗口	tokenize、head FP32 + argmax、PYNQ 驱动	稀疏脉冲适合 PL；唯一浮点段留在 PS 便于复核
LNN	CfC 单步数据通路、权重/偏置/激活 ROM、状态寄存器	ONNX 导出、定点导出、驱动、逐位比对	固定 9088 拍的单步递推；参数量小可全片上

3.2 硬件接口与地址映射

三个加速器共用同一套接口抽象：AXI4-Lite 寄存器窗口用于配置、启动与状态轮询；AXI-Full 或 DMA 用于参数与数据搬运；PL DDR4 提供大块存储。以 SNN 平台为例，PS 的 M_AXI_HPM0_LPD 经 AXI-Lite SmartConnect 接到加速器 CSR，S_AXI_HP0_FPD 经 SmartConnect 接 PL DDR4；LNN 平台由同一平台工程派生，npu_clk 为 300 MHz 单时钟域。地址映射见表 4。

表 4 地址映射与接口参数

模块 / 接口	地址或参数	说明
加速器 CSR 窗口	0x4_8000_0000（4 KiB）	AXI4-Lite，32 位寄存器访问
PL DDR4 CPU 窗口	0x4_0000_0000（1 GiB）	PS 可见的 PL 侧存储窗口，DMA 与 MMIO 共用
CNN NPU CSR / DDR	0x8000_0000 / HP0 DDR	MobileDW PS-DDR 平台映射，与其他平台不可互换
SNN 数据窗口	0x4_0000_0000 起（参数 + 输入 + SUMS）	3 路读 DMA + 1 路写 DMA
LNN 数据窗口	0x4_0000_0000 起（输入 128 B + 输出 64 B）	单次 run 写入 64 输入、读回 64 输出与末状态
约束	5 ns（200 MHz）/ 3.33 ns（300 MHz）	单时钟域，不降频、不加时序例外

3.3 软件接口

PC 侧接口统一为“一个命令 + 一个 manifest”。CNN 是 onnx_gen 与 npu_gen 两个命令行；SNN 是 toolchain 的 compile / build / package 三个子命令；LNN 是导出脚本与样例驱动。三个库的输出都把模型结构、参数清单、

RTL 源码哈希与数值合同写进 manifest，板端驱动只认 manifest 与寄存器协议。

板端接口统一为 PYNQ：加载 overlay、按寄存器协议写入配置与输入、启动、轮询 DONE、读回结果、与期望值比对并落盘。三个库的驱动都保留 source 字段，明确区分 hardware 与 simulation-model，避免把模型输出写成实测。图 5 给出软件工具链结构。

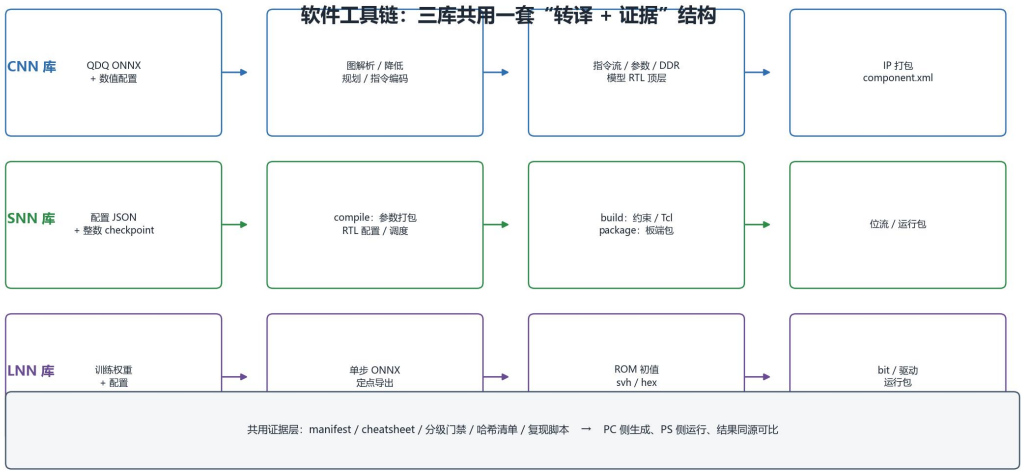


图 5 软件工具链（三库共用一套转译 + 证据结构）

3.4 时钟、复位与 CDC

CNN 与 SNN 平台使用 200 MHz（5 ns）单时钟域；LNN 使用 300 MHz（3.33 ns）。跨时钟域只出现在平台侧：PS 100 MHz 与 PL 200/300 MHz 之间的数据搬运通过 XPM 异步 FIFO 完成，残余单比特控制用 2FF 同步器并标 ASYNC_REG；不使用大范围异步时钟组例外。

复位方面，平台专用适配层由各时钟域分别复位，XPM 复位只由写时钟域驱动——早期版本把两个时钟域复位 OR 后驱动 XPM 复位，导致 48 条违例被当成同步路径处理；改为分域复位后 r5 综合 WNS 恢复到 +0.214 ns。

4 优化过程与性能资源对比

4.1 方法与迭代

优化分三个层次：算法与数值（量化、定点、LUT 化）、微架构（流水线、时分复用、引擎裁剪）、物理实现（约束、复位、实现策略）。每一层都

要求“优化前后都有数字、数字有证据、证据能复算”，不接受只给结论不给基线的改动。表 5–表 8 分别给出三个库的优化前后对比。

4.2 CNN 库优化

第一项是图感知的引擎裁剪。旧全算子平台把 13 个物理引擎全部实例化（191,542 LUT / 527 DSP）；改为按模型图决定实例后，YOLO 只保留 dense/copy/eltwise/resize/maxpool2d 5 个引擎（PHYSICAL_ENGINE_MASK=4365），平台降到 140,028 LUT / 471 DSP，LUT −26.9%、DSP −10.6%，动机是“未用核不得阻塞时序与资源”。

第二项是 MAC 后端。串行 9·Tn 累加改为平衡寄存器加法树（MAC_IMPL=2）再到双乘积/DSP48（MAC_IMPL=3），代价是卷积延迟从 9 拍变为 12/13/14 拍（Tn=4/8/16），换来时序收敛。第三项是容量推导：从 MAX_C*MAX_W 派生改为按图推导行容量与权重 tile 数，去掉未用容量。

并行度方面，8/8 历史平台端到端 239.5 ms；16/8 配置把 YOLO 端到端做到 51.4 ms（PS 等待约 20 ms + PL 推理约 31 ms），资源为 LUT 106,531 / DSP 1,203 / 功耗 7.581 W。两组不是同一位流，只作趋势对照，不作为严格 A/B。表 5、表 6 给出数据。

表 5 CNN 库优化前后对比

优化项	优化前	优化后	变化/代价
引擎实例化	全算子平台：191,542 LUT / 527 DSP	图裁剪：140,028 LUT / 471 DSP	LUT −26.9%，DSP −10.6%
MAC 后端	串行 9·Tn 累加（MAC_IMPL=0）	平衡树（2）→ 平衡树 + DSP（3）	卷积延迟 9 → 12/13/14 拍
容量推导	MAX_C*MAX_W 派生	按图推导行容量与 tile 数	去掉未用容量与 gather 网络
并行度（YOLO）	8/8 历史平台 239.5 ms	16/8 配置 51.4 ms（PS 20 + PL 31）	端到端约 4.7×；资源见下
16/8 资源	—	LUT 106,531 / DSP 1,203 / 7.581 W	DSP 增加、LUT 下降，换并行度

表 6 CNN 8/8 与 16/8 并行度对照

平台	LUT	DSP	FF	BRAM	功耗	端到端
8/8（历史 r4）	140,028	471	132,261	157.5	—	239.5 ms
16/8（新配置）	106,531	1,203	201,413	127.5	7.581 W	51.4 ms

4.3 SNN 库优化

第一项是 5 级 MAC 流水。单拍完成 64 个乘加与加法树在 200 MHz 下过不去，于是在 QKV/AV/proj/MLP 里插入 4 级寄存器：S1 操作数寄存、S2 64 乘积、S3 两个半和、S4 总和、S5 32 位累加写回。代价是每拍多 4 级排空，回放周期数上升约 28.5%，但结构与时序数值不变。

第二项是保持单元级数标定。16 级 probe 的 setup WNS 为 −0.191 ns（被否），10 级 +0.039/+0.001 ns（过门但余量过薄），最终定 8 级：setup +0.135 / hold +0.023 ns。第三项是平台 r5→r6 的 CDC 重构：r5 的 setup WNS −0.886 ns、9,728 失败端点；增加平台专用适配层、去掉 OOC 的 8 级 LUT1 链、残余 CDC 改 XPM FIFO 后，r6 达到 +0.245 / 0 失败，LUT 从 76,489 降到 72,167。

第四项是 r7 的 SUMS 读窗修复：r6 的 SUMS 只映射 32 字，每 32 通道回绕；r7 扩到 128 字（0x100–0x2FC），但 0x100>>2=64 造成的 +64 字偏移未在硬件里修，改由软件 ROT=64 补偿；代价是 WNS 从 r6 的 +0.245 降到 +0.051 ns，仍为 0 失败端点。表 7 汇总。

表 7 SNN 库优化前后对比

优化项	优化前	优化后	代价/说明
5 级 MAC 流水	e2e 2 样本 5,511,908 拍；small 31,582；real 8,267,861	e2e 7,084,772（+28.5%）；small 39,774；real 10,627,157	每拍多 4 级排空，结构与数值不变
流水级数标定	16 级：setup −0.191 ns（被否）；10 级 +0.039/+0.001	8 级：setup +0.135 / hold +0.023	每级 8 个 LUT1
平台 CDC（r5→r6）	WNS −0.886 / 9,728 失败；LUT 76,489	WNS +0.245 / 0 失败；LUT 72,167	平台适配层 + XPM FIFO
SUMS 读窗（r7）	32 字窗口，32 通道回绕	128 字窗口（0x100–0x2FC），软件 ROT=64 补偿	WNS +0.245 → +0.051，仍 0 失败

4.4 LNN 库优化

第一项是门控公式修复（V1.0→V1.1）。早期公式 gate_pre=ROUND(lin_a·Δt + bb) 漏掉 W_b z，84 个权重里 20 个 Wb 不参与，与“4 分支各 20 权”矛盾；改为 gate_pre=ROUND(lin_a·655 + (lin_b<<12)) 后，全部整数 golden、RTL 与 3 个 TB 重做并重跑，自检回归（toy 例期望 2703，旧公式只有 655）通过。

第二项是面积换延迟。若 16 组线性输出并行需要 16 个乘法器；改为 1 个 DSP48E2 串行时分复用（96 beat/步，142 拍/步、64 步 9088 拍），让 101 参数

的小网络塞进最小资源并在 300 MHz 收敛。第三项是激活函数：CORDIC/多项式改为 2×1025×int16 同步 ROM，换确定性与逐位一致性。

第四项是实现策略：默认流程改为 Performance_ExplorePostRoutePhysOpt + 路由后 phys_opt_design -directive Explore，争到 +0.2 ns 级 setup 余量，最终 WNS +0.226 / WHS +0.011，不改 RTL、不加 false path。第五项是板端部署鲁棒性：export 目录查找从“只查少量路径”改为按内容打分的多候选，并自带 board/export，现场整包拷贝即可运行。表 8 汇总。

表 8 LNN 库优化前后对比

优化项	优化前	优化后	效果
门控公式	gate_pre=ROUND(lin_a·Δt+bb)，20 个 Wb 不参与	gate_pre=ROUND(lin_a·655+(lin_b<<12))	参数全参与，自检回归通过
MAC 结构	16 组线性输出并行需 16 个乘法器	1 个 DSP48E2 串行 TDM，142 拍/步	核 DSP=1，64 步 9088 拍
激活函数	CORDIC / 多项式	2×1025×int16 同步 ROM	确定性、与参考逐位一致
实现策略	默认流程	Performance_ExplorePostRoutePhysOpt + phys_opt	WNS +0.226 / WHS +0.011
部署鲁棒性	只查少量 export 路径	内容打分多候选 + 自带 board/export	整包拷贝即可运行

4.5 三库最终指标

表 9 给出三个库的最终资源、时序与功能结果。所有数字均可在对应库的 evidence 目录或报告中回溯；未执行项明确标为 not_run。

表 9 三个库的最终指标

指标	CNN 库	SNN 库	LNN 库
资源（LUT/FF/BRAM/DSP）	140,028 / 132,261 / 157.5 / 471（r4）；106,531 / 201,413 / 127.5 / 1,203（16/8）	40,964 / 71,173 / 186.5 / 128（OOC）；72,167 / 110,508 / 227 / 131（平台 r6）	25,221 / 32,890 / 28.5 / 4；CfC 核 2,189 / 3,280 / — / 1
时序	WNS +0.075 / WHS +0.010（r4）	OOC +0.135 / +0.023；平台 r7 +0.051 / +0.011	WNS +0.226 / WHS +0.011
功能	ModelSim 15/15、DW V2 23/23、MaxPool2D 2 快照 0 mismatch	冻结 RTL 回放 0 mismatch	RTL 逐位仿真 0 mismatch
板端	历史 0.2395 s；16/8 51.4 ms	记录级 3/3 匹配	离线模型 29/29，真实板测待现场

5 大模型协作记录

5.1 协作模式与工作流

本作品采用“人工主导、智能体执行”的协作模式。项目负责人负责选题与总体方案、技术路线取舍、验收口径制定、关键节点评审、板端实验与现场反馈；大模型智能体承担工程实现与过程管理，包括任务书拆解、代码与脚本编写、仿真与综合的批量执行、证据整理与文档初稿。两者通过结构化文档交接，智能体的每一项产出均须经人工验收后才能进入下一环节。

工作流分为五步。第一，任务书：明确目标、边界、必读文档、执行步骤、验收标准与交付物；第二，执行：按任务书实施，长周期任务以后台作业编排，必要时并行分工；第三，交接：记录结论、证据路径、未闭合项与下一阶段建议；第四，门禁：仿真、时序、板端三级验证逐级出具证据；第五，沉淀：将问题与修复写入项目规范与技能包，形成下一轮任务的前置检查。该流程使智能体能够在明确约束下持续工作，同时保证方向判断与验收决策始终由人做出。

5.2 提示词与指令设计

与智能体的交互以结构化指令为主，避免开放式对话。代表性指令类型及设计要点见表 10。

表 10 提示词与指令设计

指令类型	设计要点	作用
任务书	目标、边界、必读文档、验收标准、交付物五要素齐全，并声明不得越界开展下一阶段工作	任务可执行、可验收、可交接
交接指令	要求输出结论、证据路径、未闭合项与下一任务书，并附新会话开场说明	跨会话保持上下文一致
验收口径	要求批量推进、阶段验收，每个结论必须标注 pass / fail / partial / not_run	减少无效等待，统一证据标准
纠错指令	指出具体现象并要求先解释原因、再给出修复方案	快速定位问题，防止错误结论扩散
范围控制	明确任务结束点与并行子任务的停止条件	控制并行规模，避免结果分叉

指令设计遵循三条原则：一是可执行，每条指令对应可检查的产出；二是可追溯，要求给出文件路径与执行命令；三是留痕，重要决策与结论写入交接文档，避免只存在于对话中。

5.3 自我纠错与人工把关

智能体在长周期任务中会出现判断偏差，本项目的关键纠错均由人工评审或证据复核触发。表 11 列出五类典型问题及处理过程。

表 11 典型问题、发现方式与纠正结果

问题	发现方式	纠正措施	结果
资源门槛缺乏依据	项目负责人质疑并追问数据来源	按历史运行数据重新测算并下调门槛	恢复施工，避免继续等待
核级 OOC 被当作平台完成	项目负责人追问是否连接 Block Design、能否生成位流	补做平台 Block Design、约束与全片实现	平台实现收敛，固定样本板测通过
跨时钟域复位处理不当	综合报告出现违例，证据复核时发现	复位按域拆分，XPM 复位仅由写时钟域驱动	综合时序恢复，CDC 重构后收敛
板端输入布局错误	板端比对时 6 个输出头全部不一致	定位输入排布并修正驱动	6/6 一致，板测通过
运行环境改变 manifest 哈希	复算 manifest 时发现不一致	清理缓存、固定运行参数并写入检查单	校验恢复通过

上述问题的共同点是：智能体可以完成实现与验证，但结果是否可信、是否达到验收标准仍需人工判断。项目负责人通过质疑数据来源、追问交付完整性、核对证据文件，纠正了多处偏差；这些判断随后被固化为规则，写入任务书模板与技能包。

5.4 协作成效与边界

表 12 为开发期协作规模统计，仅用于说明协作强度，不作为成果指标。

表 12 开发期协作规模统计

指标	数值	说明
开发会话 / 有效指令 / 助手消息	116 / 459 / 18,147	45 天、9 个工作目录
工具调用（含子调用）	约 2.75 万次	以命令执行、文件读写与检索为主
并行团队 / 成员 / 共享任务	17 / 68 / 71	用于并行开发与文档整理
记录在案的修复	15 类	均形成检查项或技能

边界说明：其一，统计仅覆盖本机开发日志，空会话与无关会话已剔除；其二，同一回答的模型归属无法逐条追溯；其三，板端数据由现场人员回传并与仓库文档交叉核对；其四，智能体产出均经人工验收后采信；其五，技能包为项目结束后提炼，开发期间未直接调用技能工具，这一点在技能包章节中同样如实说明。

6 技能包的提炼过程

6.1 从失败到规则

技能包不是凭空写的说明书，而是从真实失败里长出来的。表 13 给出五条“失败—观察—规则—技能”的提炼链；每条规则都能指到一次具体事故。

表 13 从失败到规则的提炼链

失败/观察	规则	落到哪个技能
把无依据的 100 GiB 当硬门槛，卡住数天	任何门槛必须给出计算或测量依据，否则标为待核实	evidence-gate：结论必须带来源
把核级 OOC 当平台完成，被追问“连 BD 都没连”	OOC 与平台是两道门，OOC 通过不等于能出 bit	cnn/snn-translate：物理门分两级
pycache 改变 manifest 哈希，板端校验失败	运行前清缓存，固定 -B；manifest 与运行环境绑定	lnn-export：运行环境固定
板端驱动只查少量 export 路径，现场报错难排查	按内容打分多候选，自带期望值目录	lnn-export：部署鲁棒性检查
把仿真模型输出写成板端实测	source 字段必须 hardware / simulation-model 二选一	evidence-gate：来源标注

6.2 技能包结构与内容

最终交付 4 个技能，放在 skill/skills/ 下，每个技能是一份 SKILL.md，包含适用、输入、步骤、验收、边界、失效条件六段。表 14 给出摘要。

表 14 四个技能包摘要

技能	输入 → 输出	验收要点	边界
cnn-translate	静态 INT8 QDQ ONNX → 指令流/参数/RTL/IP	编译退出码 0、指令数>0、IP 含 component.xml	只支持静态 shape 与 16 操作码
snn-translate	配置 + 整数 checkpoint → 参数包/RTL 配置/调度	manifest 哈希一致、回放 0 mismatch、时序 0 失败	结构参数化有边界；图像线 partial
lnn-export	训练权重 → 单步 ONNX → int16 定点 → ROM 初值	7/7 文件逐字节一致、max|Δ|≤1e-5、板端 64/64	结构固定 1→4→1；需三件套同源
evidence-gate	任意结果 → 分级结论与证据索引	每个 pass 指文件路径；无来源数字不出现	只判定不代替仿真/综合

6.3 有效性验证

有效性用三种方式验证。第一，装进下一轮任务的开场白：把规则写进任务书后，同类错误在后续会话中不再复发——资源门槛争议后没有再出现凭空门槛，OOC 教训后新增“平台门”检查项，pycache 事故后写进板端拍摄检查单。第二，可执行：4 个技能的步骤都在本仓库真实跑过，命令与期望输出可对照。第三，可交接：技能与 evidence-gate 配合后，新会话接管时先读技能再读任务书，30 秒内能判断“这件事能不能做、做到什么程度算通过”。

需要说明的是，开发期智能体的 skill 工具调用为 0 次——技能包是事后从失败中提炼的，不是开发时就在用的。这一点在报告里如实写出，也正是它的价值：它把已经踩过的坑固化成下一轮的前置检查。

6.4 边界与失效条件

· 技能只覆盖本仓库声明的模型族与数值合同，超出范围应拒绝而不是近似套用。

· 工具链或器件升级后，技能里的版本假设需要重验，否则按失效处理。

· 技能不替代仿真与综合工具，也不替代人工验收；它只保证步骤、检查项与证据格式一致。

7 复现说明

7.1 环境要求

复现需要一台 Windows 主机（转译、仿真、Vivado）与一块 ZU15EG 开发板（板端运行）。工具版本与器件信息见表 15；版本不一致时按各库 README 的兼容说明处理，不保证跨版本结果逐位一致。

表 15 复现环境

项目	版本/型号	用途
开发板	RIGUKE ZU15EG；AMD xczu15eg-ffvb1156-2-i	PL 加速器与 PS 运行
PS 系统	PYNQ Linux（PYNQ 2.7）	板端 Python 运行时
Vivado	2021.1	综合、实现、位流
仿真	ModelSim SE-64 2020.4	RTL 回放
Python	3.9（miniconda）	工具链与脚本
依赖	numpy、onnx、onnxruntime、torch（CPU）	转译与参考计算

7.2 从零开始步骤

以下步骤以提交包为根目录；每一步都给出命令与期望结果，任一步失败先按 7.3 排查，不要跳过。

步骤 1：检查目录与版本

```
cd <project_root>
python -c "import sys; print(sys.version)"
vivado -version       # 期望 Vivado v2021.1
```

步骤 2：CNN 转译与 IP 打包

```
cd src/cnn
```

```
python -m tools.onnx_gen --onnx <model.onnx> --out out/cnn --tn 4 --tm 4 --mac-impl 3
python -m tools.npu_gen --generated out/cnn/generated --out out/cnn/ip_repo
```

期望：编译器退出码 0，compile_manifest.json 含 instruction_count>0 与源码哈希；ip_repo 下出现 component.xml。

步骤 3：SNN 转译与回放

```
cd src/snn
python -m tools.toolchain --backend spikformer compile --config <cfg.json> --weights <ckpt.npz> --out out/snn
```

期望：54 个参数 blob、28 份 RTL 源哈希、manifest_sha256 生成；随后按 build --stage sim 跑 RTL 回放，期望 mismatch=0。

步骤 4：LNN 导出与对拍

```
cd src/lnn
python user/export_demo.py --out-dir user/outputs/repro01
```

期望：7/7 文件逐字节一致，RESULT: PASS；ONNX vs PyTorch max|Δ| ≤ 1e-5。

步骤 5：仿真门禁

```
cd sim/cnn && ./run_modelsim_replay.ps1    # 期望 0 mismatch
cd sim/snn && ./run_replay.ps1             # 期望 0 mismatch
```

步骤 6：物理门禁（Vivado）

```
cd build/snn
vivado -mode batch -source build_ooc.tcl    # OOC：0 失败端点
vivado -mode batch -source build_platform.tcl # 平台：0 失败端点，生成 bit/hwh
```

期望：OOC WNS +0.135 / WHS +0.023；平台 r7 WNS +0.051 / WHS +0.011；bit/hwh/xsa 三件套哈希配对。

步骤 7：板端运行

```
scp -r board/lnn xilinx@<board_ip>:/home/xilinx/lnn
ssh xilinx@<board_ip> 'cd /home/xilinx/lnn && python3 -B lnn_cfc_run.py --sample sample_000 --checks'
```

期望：source=hardware、outputs matched 64/64、final_state_match=True、RESULT: PASS；SNN 板端按 board/snn/README 运行 demo_nlp.py。

步骤 8：证据核对

按表 17 逐项核对：manifest 哈希、回放 0 mismatch、时序 0 失败、板端 64/64 或 3/3，全部通过后再写报告。

7.3 常见问题

表 16 复现常见问题

现象	原因	处理
板端报 export/manifest.json config/hash check failed	Python 写了 __pycache__，与 manifest 记录的哈希不符	运行前删 export/__pycache__，统一加 -B
OOC 通过但平台不收敛	OOC 不包含 BD、DMA、时钟网络	按平台工程重跑，分开记录 OOC 与平台门
驱动报 export/ directory not found	只拷了驱动/位流，没带期望值目录	整包拷贝 board/；或设 CFC_EXPECT_DIR
complex 模型转译时 shape 推导失败	动态 shape 或超出 16 操作码	改静态导出；不支持即拒绝，不静默回退
时序报告与旧结果不一致	实现策略或 Vivado 版本不同	记录工具版本与实现策略，按同版本对比
板端数字与报告不一致	把仿真模型或历史记录当本次实测	核对 source 字段与证据路径，重跑后更新

7.4 证据核对清单

表 17 是交付前的核对清单，逐项打勾后才允许提交。

表 17 证据核对清单

环节	核对项	期望	证据位置
转译	manifest 哈希自洽	编译退出码 0、哈希一致	转译输出目录的 manifest
仿真	RTL 回放 mismatch	0	sim/*/evidence/*.json
时序	OOC / 平台失败端点	0 / 0	build/*/reports/*timing*
位流	bit/hwh/xsa 配对	三件套同源、哈希一致	board/*/MANIFEST 或 SHA256SUMS
板端	输出与参考比对	CNN 51.4 ms；SNN 3/3；LNN 64/64	board/*/results/*.json
文档	结论与证据对应	每个 pass 有路径	report/ 与 evidence/

附录 A 工程仓库目录结构（3.3.5.4 映射）

提交包按竞赛指南 3.3.5.4 的推荐结构组织：仓库内所有目录与文件名使用纯英文，中文只出现在文档正文与说明里。下表给出结构与内容映射；开发期的临时目录、会话日志、视频素材、构建备份均不进入提交包。

```
<project_root>/
├── README.md        # 项目简介 + 复现入口 + 边界说明
├── src/             # 设计源码（RTL / 工具链 / PS 软件）
│   ├── cnn/         # ONNX 编译器、IP 打包器、RTL 算子库
│   ├── snn/         # Spikformer 工具链（compile/build/package）与 RTL
│   └── lnn/         # CfC 导出器、RTL 与驱动
├── sim/             # 仿真与回放：ModelSim 脚本、TB、回放证据
│   ├── cnn/  snn/  lnn/
├── build/           # 可复现构建：综合/实现 Tcl、OOC 与平台报告
├── board/           # 板端工程与实测输出：bit/hwh/xsa、驱动、summary
├── data/            # 测试数据与参考结果：配置、样例、期望值
├── skill/           # 技能包：README + 4 个 SKILL.md
└── report/          # 设计报告、作品说明、大模型协作记录
```

表 18 给出每个目录与原始工程内容的对应关系，以及为什么这样放。

表 18 目录映射与放置理由

目录	内容来源	放置理由
src/cnn	DL/rtl、DL/tools、DL/docs	设计源码与生成器，复现需要
src/snn	SNN/rtl、SNN/tools、SNN/docs	同上；工具链三段式入口
src/lnn	LNN/rtl、LNN/docs	同上；CfC 模块与接口说明
sim	DL/verification、SNN/verification、SNN/evidence、LNN/verification、LNN/evidence	仿真脚本与回放证据
build	DL/scripts + Tcl/PS 脚本与报告、SNN/scripts + build 清单、LNN/scripts + 验收报告	构建入口；构建产物不重复放大文件
board	CNN 板端包（位流 + runtime）、SNN/board、LNN/board	板端工程与实测输出
data	CNN 样例 JSON、SNN/examples、LNN/export	配置、样例与期望值
skill	本次提炼	技能包与检查表
report	本次撰写	设计报告、作品说明、协作记录

明确排除：开发期的 _report_work（会话分析、解压日志）、video（视频素材）、图片（手机照片原图）、DL/examples 中的大型训练产物、Vivado 运行日志与备份、各目录的 __pycache__。

附录 B 缩略语

CNN Convolutional Neural Network 卷积神经网络
SNN Spiking Neural Network 脉冲神经网络
LNN Liquid Neural Network 液态神经网络

CfC Closed-form Continuous-time 闭式连续时间网络
ONNX Open Neural Network Exchange 开放神经网络交换格式
QDQ Quantize-Dequantize 量化-反量化节点
MAC Multiply-Accumulate 乘加运算
OOC Out-of-Context 脱离上下文的独立综合/实现
CSR Control and Status Register 控制状态寄存器
DMA Direct Memory Access 直接存储器访问
LUT Look-Up Table 查找表
WNS Worst Negative Slack 最差建立时间裕量
WHS Worst Hold Slack 最差保持时间裕量